FLOW GRAPH REPRESENTATION

Alex Orailoglu

Gould Research Center
40 Gould Center
Rolling Meadows, Illinois 60008

Daniel D Gajski

University of Illinois
Department of Computer Science
1304 W. Springfield Avenue
Urbana, Ill. 61801

1. INTRODUCTION

Methodologies based on simple components, such as gate arrays and standard cells, are not adequate when designing complex VLSI systems. Silicon compilation, an evolutionary step from standard cell methodology, offers an increase in design complexity with an increase in design productivity. Silicon compilers can be broadly divided into structural, functional, and intelligent silicon compilers ([Gajs85]). In *structural silicon compilation,* the designer explicitly defines the microarchitecture; i.e. a structure consisting of registers, busses, RAMs and ALUs. *Functional silicon compilers* transform a behavioral description into a microarchitecture automatically.

All silicon compilers can be distinguished broadly by the detail in the input description and the model of the target architecture. The input description detail is characterized by bindings of variables to storage elements (registers and memories), operations to functional units (ALUs, multipliers, shifters, etc.) and register transfers to control states. In structural silicon compilers all of the three bindings must be specified explicitly by the designer. Functional silicon compilers do not require all three bindings to be specified explicitly ([Park84]). SILC ([BlFR85]), and MacPitts ([Sout83]) specify states and registers but allow the compiler to allocate functional units. Some compilers in this class allow functional units of different type and within the same type they allow different cost/performance options ([BlFR85]). DAA does not require any binding to be specified ([KoTh85]). When no binding is specified, the compiler may use a fixed algorithm to translate the input description to the target architecture. *Intelligent silicon compilers* are capable of making tradeoffs and tuning the design to user constraints in area, time, and power. This is achieved by using a flexible, intermediate representation that will allow tradeoff analysis. The data flow graph is such a representation that exposes maximum parallelism in the input description ([Camp85]). However, it is not sufficient, since it does not show control precedences.

In functional silicon compilation, a dataflow representation can be mapped to one of these three target architecture models: datapath, control unit and datapath, control unit and datapath with memory. The first model is very popular with commercial silicon compilers, such as Genesil or Concorde. The second model is used by most silicon compiler projects, such as MacPitts ([Sout83]), SILC ([BlFR85]), APOLLON ([JaJe85]), and ALGIC ([JoGl85]). The third model has not been used at all. One problem is in defining an adequate memory model that will produce a good design without complicating compiler structure. Secondly a sophisticated data flow analysis needs to be performed to allow for tradeoffs involving memory references. Sometimes, even data flow analysis cannot resolve dependencies involving memory references at compile time.

In this paper, we describe such a representation that uses both data and control dependencies. In case when data dependencies cannot be established, we preserve control dependencies implied in the semantics of the behavioral description. This preservation is necessary for both memory and register types of storage operations. Thus area/performance tradeoffs can be done whenever possible. Furthermore, we describe compiler algorithms for converting the behavioral description into this representation. The behavioral description to be examined includes constructs for loops (FOR, UNTIL, WHILE), conditionals (IF), memory references and arithmetic and logical operators.

The rest of this paper is laid out as follows: In Section 2, we intuitively describe our control/data flow representation. We describe control flow structure in detail in Section 3, while data flow structure is described in Section 4. Section 5 sketches compilation algorithms. Section 6 summarizes the contributions of this paper.

2. CONTROL vs. DATA FLOW

The control/data flow representation is made up of two parts. The data flow graph representation can be used to establish area/performance bounds and make area/performance tradeoffs. A control flow graph is used to generate a Control Unit design. It is preferrable to represent constructs in data flow, as far as possible, since the data flow representation does not force the creation of a new state in the Control Unit.

The input behavioral constructs are analyzed and decomposed into a number of blocks. Each block is a node in the control flow graph which shows the sequence of executions between different data flow blocks. It also models execution choices in conditional or loop statements.

A number of language constructs are always represented in control flow. All three types of loop constructs, FOR, WHILE, and UNTIL statements, and procedure calls are represented in control flow. If any of the above constructs are nested within other statements, the nesting statements themselves are decomposed into a control flow representation.

0738-100X/86/0000/0503$01.00 ©1986 IEEE

Control flow nodes can point to a corresponding data flow representation of the block. A data flow graph shows clearly the data dependencies that exist in a block of assignment or conditional statements. This representation enables the silicon compiler to easily find out information about how to best partition the program into control steps and which functional units to assign to each operation so that performance and area goals are met. The majority of the nodes in a data flow graph represent arithmetic or logical operations.

Conditional statements, can be represented either in control flow or data flow. If any of the branches of a conditional embed a construct that needs to be represented in control flow, then the conditional itself needs to be represented in control flow, since control flow nodes cannot be embedded within a data flow graph. Some array operation sequences force an embedding conditional into a control flow decomposition. In that case, each of the control flow branches is made up of a single data flow block. Fig. 1 shows a conditional with array references represented in control flow,

```
case C of
      1:  X=X+2
          A[3]=X+5
      2:  A[I]=X+3
          Z=A[2]+7
   ELSE:  A[2]=X+W
end case;
```

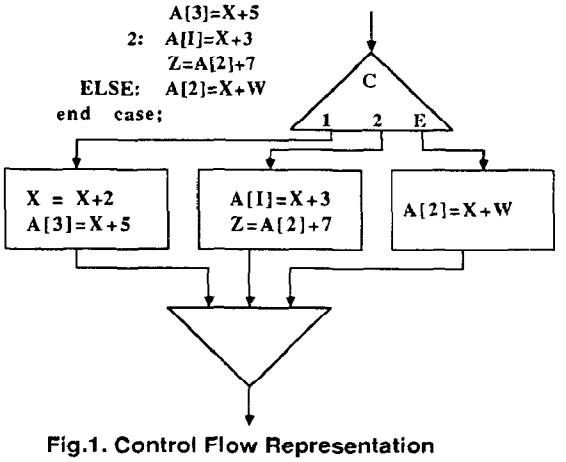

Fig.1. Control Flow Representation

whereas Fig. 2 shows another conditional with array references represented in data flow.

Even when a construct is to be represented in data flow, there are cases when pure data dependencies are not adequate in representing correct program semantics. This is the case with representation of storage operations, particularly array

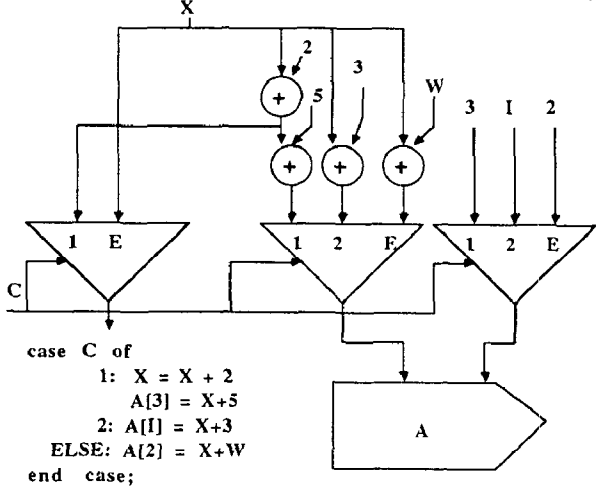

Fig.2. Data Flow Representation

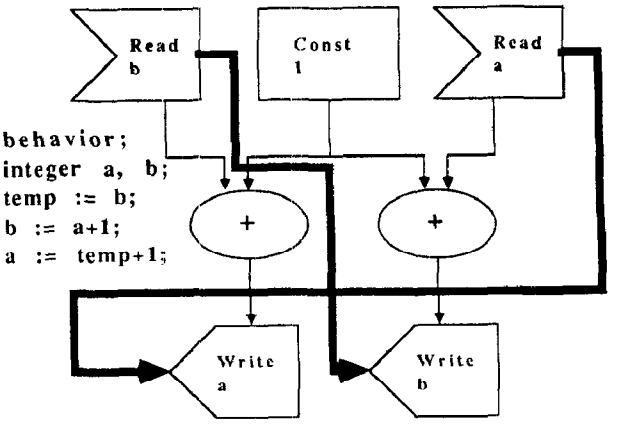

Fig.3. Control Precedences in Dataflow

references. The reason for this is that at compile time it is often not obvious which memory location is accessed or overwritten. As a result, we impose extra control dependencies within the data flow representation to ensure operations will be performed in an order that preserves the semantic meaning. In Fig. 3, we see a program whose semantic meaning is not preserved by pure data flow precedences. In this example, both operations have to be executed in parallel to preserve the intent of the program. If only one adder is available, though, then a temporary register needs to be introduced to avoid overwriting the register that has not yet been read. Yet there is no dependency in the data flow representation that will force the allocation of the temporary register. Control precedences (shown as heavy checked lines in Fig. 3) would force the correct hardware allocation decisions.

This example can easily be generalized to one with array references.

3. CONTROL–FLOW STRUCTURE

The control flow graph consists of several node types (Fig. 4).

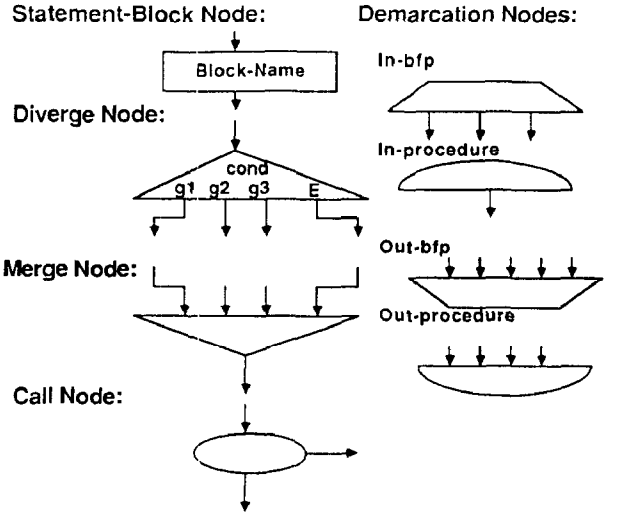

Fig.4. Control Flow Elements

The *Statement–Block* node represents a number of assignment or conditional statements. Statement–Block nodes do not follow or precede other Statement–Block nodes. They are the only control flow nodes that have a corresponding data flow block representation.

Diverge nodes distribute control among a number of control flow node sequences. Conditional statements, like IF-THEN-ELSE and CASE constructs, are represented by Diverge nodes. Loop statements give rise to Diverge nodes when represented in control flow. The control flow representation for the FOR and WHILE statements is largely similar. The control flow for an UNTIL statement is represented differently since the termination check needs to be executed after execution of the statements in the loop body.

All Diverge nodes share a common way of modelling the conditional test. The *condition variable* is always a boolean variable, except in a CASE statement, when it is an integer variable. Each of the condition statements has a number of *condition guards* against which the condition variable is tested. These condition guards are always constants. For each of these constant values, there exists a corresponding output branch. This corresponding output branch will be executed, when the value of the condition variable equals the constant value.

The number of output branches in a Diverge node is always one more than the corresponding number of condition guards. The last output branch denotes the execution path when none of the values of the condition guards matches the value of the condition variable. This path represents the ELSE part of an IF-THEN-ELSE or of a CASE statement, or the exit path from a loop statement. These divergent paths merge into a *Merge* control flow graph node. For each Diverge node, there exists a corresponding Merge node. The Merge node has as many incoming branches as there are outgoing branches in the corresponding Diverge node. Conditional statements without an explicit ELSE part have an extra branch which connects directly the ELSE branch of the Diverge node to the Merge node.

Parameterless procedure calls are represented by the *Call* node. This node has one input and two outputs. The first output points to the beginning of a procedure. The second output is the control flow node to be executed when the procedure returns.

There are four different *demarcation* nodes. Two of these are used to mark off behavioral programs. These are the *In-Bfp* and *Out-Bfp* nodes. The other two are used to mark off procedures. These are the *In-Procedure* and *Out-Procedure* nodes. The In-Bfp and In-Procedure blocks have no inputs. Whereas the In-Procedure has only one output, the In-Bfp has a number of outputs. The first one corresponds to the beginning of the control flow sequence within the program; the rest point to In-Procedure nodes, corresponding to procedures used in the block. The Out-Bfp and Out-Procedure blocks have no outputs, but a number of inputs.

4. DATA-FLOW STRUCTURE

There exist five different types of data flow nodes (Fig. 5).

Operation nodes represent arithmetic or logical operations. These nodes have at most two inputs, since all non-unary arithmetic and logical operations are converted into binary operations.

An assignment to a variable in any branch of a conditional statement gives rise to a *Choose-Value* node for that variable, when the conditional is represented in data flow. A Choose-Value node has three types of inputs and a single output. The first type of input is a single data input that is called the *condition variable*. Each node contains constant

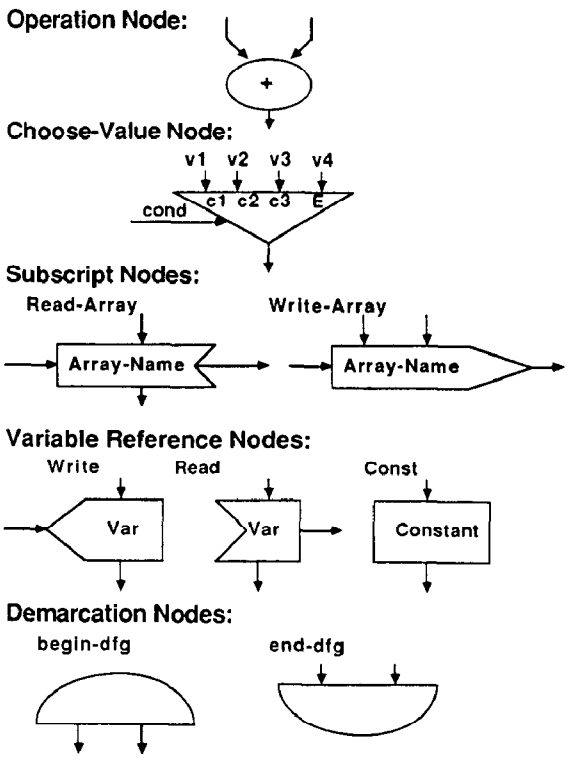

Fig.5. Data Flow Elements

values called the *condition guards* against which the condition variable is tested. These constants are booleans for IF statements, but integers for CASE statements; they make up the second type of inputs. There are as many condition guard inputs to a Choose-Value node as there are non-ELSE branches of a conditional. The third type of input is a number of data values. Each of these inputs represents the value to be propagated if the corresponding condition guard is equal to the value of the condition variable. There is always one more data value than the number of condition guards. The extra value corresponds to the ELSE case.

Subscript nodes are used to model array references. Control dependency arcs for subscript nodes are used to connect together references to the same array. The control dependencies are used to preserve a partial execution order in memory references, implied by the semantics, but not captured through data dependencies.

A *Read-Array* node takes two types of inputs. The first type of input is a number of control dependence arcs from other Subscript nodes. The second input is an index value. A Read-Array node has two types of outputs. The first is a single data output and represents the value read from memory. The second one is an outgoing control dependence arc. A Read-Array node is generated for every read to an array element, for which element a current value does not exist in the data flow graph.

A *Write-Array* node takes three different types of inputs. The first type of input is a number of control dependence arcs from Subscript nodes. The second input is an index value. The third is the data value to be stored. The output of this node is a control dependence arc. A Write-Array node is generated for every VARIABLE-indexed array operation or for

the latest value of array elements prior to a VARIABLE-indexed array operation.

There are two types of *Variable Reference* Nodes. Block input variables are represented by *Read* nodes. This node has one input and two outputs. The input arc is a demarcation arc. The first output is a data output and represents the value read. The second output is a control dependence arc to a Write node of the same variable.

Block output variables are represented by *Write* nodes. This node has one demarcation output and two inputs. One input arc represents the data value being written. The second input represents the control dependence arc from the Read node of the variable. Each constant used in a block gives rise to a *Const* node. This node has one data output and one demarcation input.

Demarcation Nodes mark off the beginning and end of a data flow graph block. The start of the block is represented by the *begin-dfg* node; the end of the block is represented by the *end-dfg* node.

5. COMPILATION ALGORITHM

A program consists of operators that act on values to create new values that are possibly stored. The data flow graph generation algorithm converts this representation into one made up of nodes and arcs. This is accomplished by converting operators to nodes and by representing assignments implicitly through data flow arcs.

The basic algorithm for data flow graph generation propagates values that correspond to variables or array elements. To achieve this, each statement is examined sequentially. At each point of the compilation, a graph exists which represents the behavioral program segment examined so far. A new subgraph is built for the right hand side of the current statement. Outgoing arcs of the graph are connected to incoming arcs of the new subgraph for the same variable, thus joining the two. The outgoing arc of the new subgraph is marked as the latest value that corresponds to the left-hand side variable of the current statement. Constant nodes are generated for constant references.

For more complex statements, such as conditionals, this basic algorithm needs to be augmented. A Choose-Value node is used for each variable that is updated within at least one branch of the conditional statement.

The data flow representation of a conditional is constructed in a bottom-up manner. The basic data flow generation algorithm for conditionals constructs a separate graph for each conditional branch. When the conditional has been analyzed, these graphs are unified into one.

Two primitive operations are used to connect together these graphs. The *Connect* algorithm connects the values used in a conditional branch to the current values of the preceding block. The *Merge* algorithm produces a flowgraph representation for variables updated in the conditional branches.

These algorithms are invoked after a separate block has been created for each conditional branch. Uses of variables in a block corresponding to a conditional branch are tied to the latest generation of a value for that variable in the block preceding the conditional, by the *Connect* algorithm.

At this point, the *Merge* algorithm is invoked on all the conditional branch blocks together with the block that corresponds to the statements prior to the conditional. All blocks are examined simultaneously so that the number of Choose-Value nodes generated is minimized.

If a conditional construct must be represented in control flow, each of the blocks is ready to be generated as a separate data flow block that corresponds to a control flow Statement-Block node. The algorithm needs to represent correctly possible multiple control flow nodes within each branch of the Diverge node in the control flow graph.

An embedded control flow conditional forces all the nesting conditional statements to be represented in control flow. A decision to represent a subpart of the program in control flow propagates itself to all nesting constructs; as a result, this attribute is a synthesized one. The decision to represent a conditional in control flow is not propagated to subsequent conditionals in the same or lower nesting levels.

Data flow representation of array assignments in conditional branches is represented as Write-Array nodes with Choose-Value nodes as both data and index inputs. This construct merges one array assignment from each conditional branch, irrespective of subscript value. Choose-Value nodes for data and index inputs are generated by the algorithm. Fig. 2 shows an example of a data flow representation while Fig. 1 shows an example of control representation of array references in a conditional statement. A control flow representation is generated, if there is an array operation following a VARIABLE-indexed Write-Array node, or if there is a Write-Array node followed by a VARIABLE-indexed array operation.

Loop statements, like FOR, WHILE, and UNTIL, result in complex control flow structures. Index incrementation, testing and initializing have to be automatically generated for a FOR statement. Statements for initialization are added at the end of the Statement-Block node that represents the statements that precede the FOR statement. Statements for incrementation are added at the end of the last control flow node within the loop body. A separate Statement-Block control flow node is used to model the termination check of loops. In a FOR statement, that node is generated automatically.

5.1. Objectives of the Array Algorithm

We will describe a procedure in this paper which incorporates memory references into the flowgraph generation algorithm. The two objectives of this procedure are to minimize the number of array access nodes in the flowgraph and to minimize the length of the critical path in control dependency chains in subscript nodes.

The benefit of the first objective lies in avoiding the penalty associated with array references; in a single port memory, a clock cycle can accommodate a READ or a WRITE operation only. Memory operations are expensive and detecting and eliminating unnecessary memory references improves the performance of the generated design.

Executing the array operations in a sequential specification order preserves the semantics of the behavioral description. Yet this representation results in one single control dependency path, which is as long as the number of subscript nodes generated for that array. By doing a more

detailed analysis of the effects of the array operations, semantics of the description can be preserved, while minimizing the length of the critical path of the control dependencies. Minimizing this critical path affords greater flexibility in design, which can be exploited by an intelligent silicon compiler.

This second objective is achieved by exposing the parallelism inherent in array manipulations. This parallelism can subsequently be exploited in two ways. In a local sense, it can be exploited in mapping array operations into microinstructions with underutilized memory ports. In a global sense, this refined representation can help an intelligent silicon compiler decide as to the best structure of memory for the requirements. Possible such choices would be number of ports/memory, or breaking up the array into pieces to be mapped into smaller memories, each with their own memory access hardware.

5.2. Reduction of Subscript Nodes

The algorithm reduces the number of array reference nodes generated by postponing the generation of Write–Array nodes, until necessary. Thus, if the array location is subsequently overwritten, it is frequently possible to forego the generation of both memory references.

The algorithm employs different methods for constant and variable subscripts. Nodes with variable array subscripts are minimized in an optimization phase after graph generation, while constant subscripts are treated during data flow generation.

The procedure for minimizing Subscript nodes for array references with constant indices is shown graphically in Fig. 6. The procedure needs to construct two data structures for two tasks. When an array value is needed, the algorithm needs to determine the arc that corresponds to the latest value for that array element. (This can be the outgoing arc of an Operation or of a Subscript node.) This arc, if it exists, is used during flowgraph generation, instead of generating a new subscript node. Similarly, the algorithm needs to separately determine the latest value for array elements for which no subscript node has yet been generated and be able to distinguish it from values of array elements for which a subscript node has been generated.

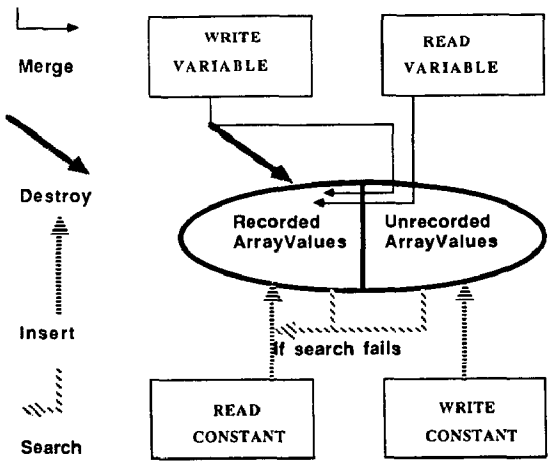

Fig.6. Algorithm for generation of Subscript Nodes and reduction of Nodes with Constant Indices

The procedure makes its decisions on whether the current array operation is a READ or a WRITE and whether the index of the array operation is a VARIABLE or a CONSTANT. The current array operation types are shown as rectangles in the figure. We show the two data structures in ovals. One of these data structures *(RecordedArrayValues)* keeps track of the most recent value of the array element, for which a subscript node has been generated. The second data structure, *(UnrecordedArrayValues)* keeps track of the latest value of a array element for which no subscript node has yet been generated. (Both of these data structures internally store dataflow arcs that correspond to the values; we will instead show the corresponding expressions in the following examples.) The two data structures are jointly used to determine if a flowgraph arc exists that corresponds to a required array value.

Three types of operations are used to update these data structures, while the *search* operation is used to inquire if an array element exists in the data structures. The two data structures can be *merged,* resulting in the generation of subscript nodes for the array elements in the UnrecordedArrayValues data structure. The values in the data structure can be *destroyed.* Finally, a data value can be *inserted.*

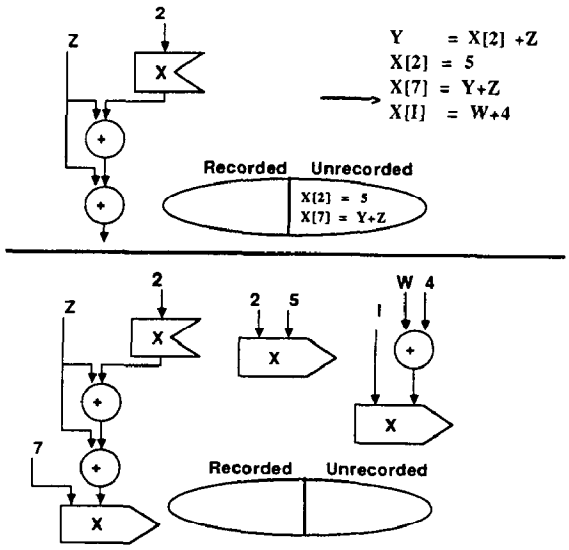

Fig.7. Flowgraph before and after analysis of VARIABLE-indexed array assignment

As an example, Fig. 7 shows the transformations and the associated data structures, when a VARIABLE–indexed array assignment is encountered. According to the procedure in Fig. 8, this results in merging the two data structures by generating subscript nodes in the dataflow graph for the array elements in the UnrecordedArrayValues data structure. This is followed by the destruction of all the current values in both data structures. A subscript node is then generated for the VARIABLE–indexed array assignment.

An optimization phase was chosen for array references with variable subscripts since this approach allows us to use the flowgraph itself to reduce the number of nodes generated. Storage requirements for the data structures of the flowgraph generator are also reduced since there is no need to track the current values of array elements with variable subscripts.

The optimization rules that are used will be shown graphically in the form of left-hand and right-hand side templates. When the left-hand side is matched, it is replaced by an instance of the template on the right-hand side. The optimization rule in Fig. 8 is used to reduce the number of Read-Array nodes with VARIABLE indices, while the template

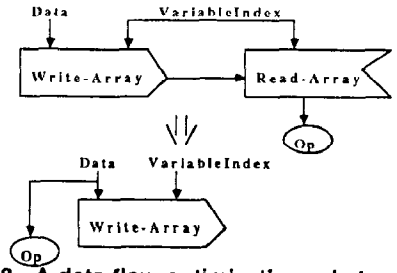

Fig.8. A data flow optimization rule to reduce Read-Array nodes with Variable Indices

to reduce the number of Write-Array nodes with VARIABLE indices is shown in Fig. 9. For this rule to be invoked, the shaded configuration should be absent from the data flow subgraph under consideration.

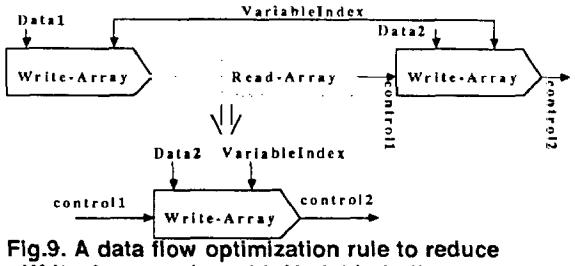

Fig.9. A data flow optimization rule to reduce Write-Array nodes with Variable Indices

5.3. Control Dependencies for Subscript Nodes

With no analysis, memory references will be executed sequentially as specified by the program. Such a rigid interpretation of program execution order can result in inefficient design decisions, since no advantage can be taken of the inherent algorithmic flexibility. To take advantage of the flexibility in the description, the possible parallelism in the execution of array operations has to be exposed.

The basic algorithm for establishing control dependencies and reducing the path length of control dependency chains is the *EstablishConnections* procedure. This procedure makes its decision based on whether the current array operation is a READ or a WRITE, and whether the index is a VARIABLE or a CONSTANT. The current array operation is then compared against all array operations that are still active, based on the same criteria of READ/WRITE, and VARIABLE/CONSTANT.

This procedure is shown graphically in Fig. 10. The boxes

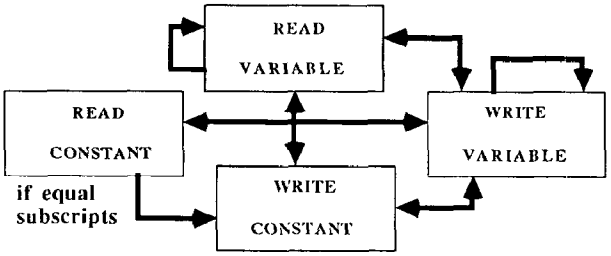

Fig.10. EstablishConnections Procedure

in the graph show the type of array node currently considered. The arrows denote that a control dependency arc is to be established between the types of nodes during flowgraph generation. Thus a control dependency should be established if a READ node with a CONSTANT index is followed by a WRITE node with a CONSTANT index, as long as the two constant subscripts are the same. Furthermore, no control dependency is established from a WRITE node with a CONSTANT index to a READ node with a CONSTANT index.

In the example in Fig. 11, prior to the analysis of the last statement, only one Read-Array node had been generated, while there were two unrecorded array values. According to the procedure of Fig. 6, subscript nodes are generated for X[2], X[7], and X[I]. The *EstablishConnections* algorithm connects a Read-Array node with a Constant index to a Write-Array node with a Constant index only if the subscripts are equal. Thus the Read-Array node in the graph of Fig. 11 is connected through a control arc to the Write-Array node with subscript 2 only; the Write-Array node with subscript 7 remains unconnected. Both Write-Array nodes are connected through control arcs to the VARIABLE-indexed Write-Array node. A control connection is also made between the Read-Array node and the VARIABLE-indexed Write-Array node.

Examining all previously generated array nodes for every new array node encountered results in a quadratic algorithm for establishing control dependencies. A number of unnecessary control dependencies are also introduced in the process, unless the analysis is sharpened. To make the flowgraph generation program more efficient, we need analyze when an array node ceases to be *active*. Nodes that are not active need no longer be considered for establishing control dependencies. x is *not active*, if all nodes, y, that depend on it would also need to depend on a node, z, which depends on x.

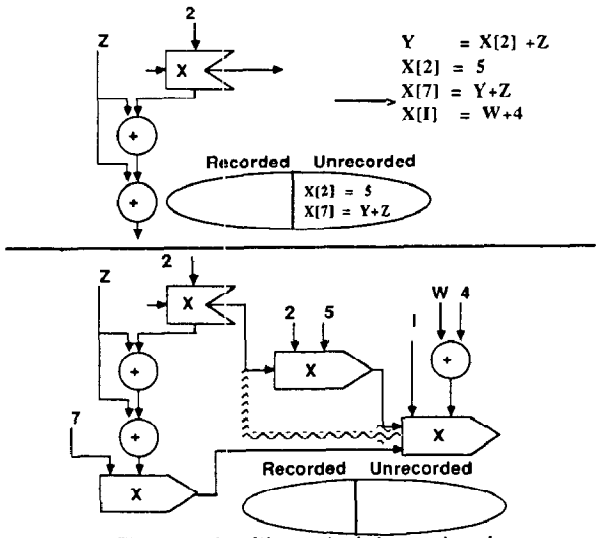

Fig.11. Flowgraph with control dependencies before and after analysis of a VARIABLE-indexed array assignment

The *KillConnectivity* procedure, which determines array nodes that are no longer active, is shown in Fig. 12. An arrow in this graph implies that if we are currently generating a node of the type that the arrow is pointing to and if there exists an *active* node of the type that the arrow is coming from, then the active node is converted into one that is *not active*. For

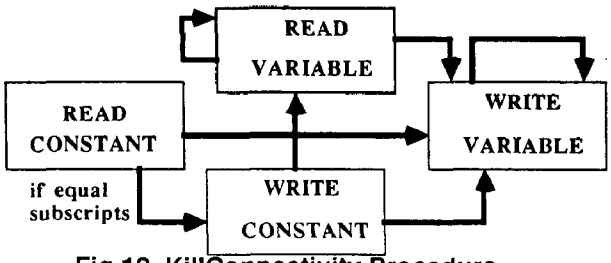

Fig.12. KillConnectivity Procedure

example, if a WRITE node with a VARIABLE index is encountered, all previous READ nodes with CONSTANT indices are marked as no longer active. In the converse sequence, there is no effect on the WRITE nodes with VARIABLE indices.

In Fig. 13, generation of the Write-Array node with subscript 2 converts the Read-Array node to not active. Prior to generating Write-Array subscript nodes, the flowgraph is shown in Fig. 13. Since the Read-Array node is no longer active, it is not considered for establishing a control dependency to the Variable-Indexed Write-Array node, thus resulting in the subgraph of Fig. 13. Fig. 13 no longer shows the extraneous control dependency arc shown in heavy lines in Fig. 11.

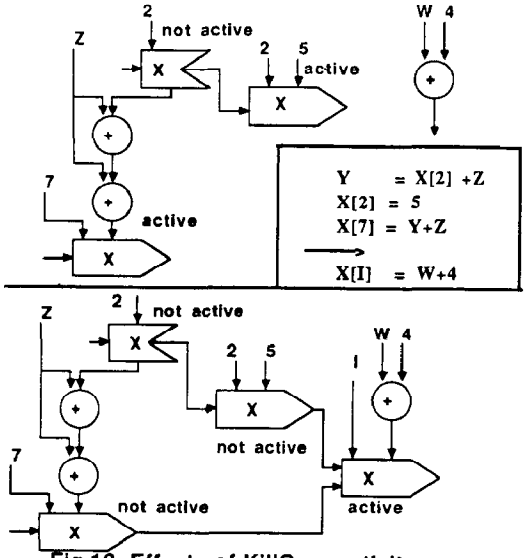

Fig.13. Effects of KillConnectivity on control dependencies for VARIABLE-indexed Write-Array nodes

Array assignments within a conditional are represented by having two separate Merge nodes, as shown in Fig. 2. Treating this Write-Array node as a VARIABLE-indexed Write-Array node ensures that the control dependencies generated are correct.

6. CONCLUSIONS

Data flow representations are powerful intermediate forms for functional silicon compilation. Although data flow models have been used for functional silicon compilation, no models exist that handle memory references. We described a hybrid data flow/control flow representation that models memory references. A compilation algorithm that generates this data flow model has been outlined. This algorithm reduces the number of subscript nodes and their control interdependencies, so that an intelligent silicon compiler can do an effective tradeoff analysis in logic synthesis. The main features of the compilation algorithm presented are:

(1) Analysis of and reduction of memory references,

(2) A hybrid representation within data flow to capture correct semantics for storage operations,

(3) Postponing the generation of subscript nodes and associated control dependency arcs to produce a reduced set of nodes that captures the parallelism in array operations,

(4) Optimizing this representation for VARIABLE-indexed array operations in a post-flowgraph optimization phase.

The flowgraph generation algorithm with memory references has been implemented as part of DESCART ([OGKC86]), an intelligent functional silicon compiler developed at Gould Research Center. DESCART currently receives two sources of inputs: a behavioral description in TI HDL ([TIHD84]), and a set of constraints to generate a design that obeys designer-specified goals for area, performance, number of pins, and power loads. DESCART maps the behavioral specifications into a microarchitecture, (and subsequently to a gate level representation), by automatically mapping variables to storage elements, operations to functional units, and register transfers to control states.

7. ACKNOWLEDGEMENTS

This research was supported through funding from US Army, LABCOM, Ft. Monmouth, N.J., under Contract Number DAAK20-84-C-0413 and Gould Foundation. We gratefully acknowledge their support and assistance.

8. REFERENCES

[BlFR85] Blackman, T., Fox, J., Rosebrugh, C., "The Silc Silicon Compiler: Language and Features", *DAC*, 1985, pp. 232-237.

[Camp85] Camposano, R., "Synthesis Techniques for Digital Systems Design", *DAC*, 1985, pp. 475-481.

[Gajs85] Gajski, D.D., "Silicon Compilation", *VLSI Design*, Nov. 1985, pp. 48-64.

[JaJe85] Jamier, R., Jerraya, A.A., "APOLLON: A Data-Path Silicon Compiler", *IEEE Circuits and Devices*, May 1985, pp. 6-14.

[JoGl85] Joepen, H., Glesner, M., "Architecture Construction for a General Silicon Compiler System", *ICCD*, 1985, pp. 312-316.

[KoTh85] Kowalski, T.J., Thomas, D.E., "The VLSI Design Automation Assistant: What's in a Knowledge Base", *DAC*, 1985, pp. 252-258.

[Park84] Parker, A.P., "Automated Synthesis of Digital Systems", *IEEE Design and Test of Computers*, Nov. 1984, pp. 75-81.

[OGKC85] Orailoglu, A., Gajski, D.D., Kuhn, R.H., Chafetz, B., "DESCART: An Expert Silicon Compiler for Fast Prototyping", *GOMAC*, November 1985, pp. 125-128.

[Sout83] Southard, J.R., "MacPitts: An Approach to Silicon Compilation", *IEEE Computer*, Dec. 1983, pp. 74-82.

[TIHD84] Texas Instruments, *VHSIC Phase 1 Program: HDL User's Manual*, June 1, 1984.